

PROCESAMIENTO DE DATOS MASIVOS

GraphRAG



Qué haremos hoy



1. Repaso rápido

RAG y grafos como herramientas de recuperación



3. GraphRAG

En qué se basa y cómo se construye el grafo de conocimiento



5. MillenniumDB

Sintaxis y operaciones: tensores, HNSW y patrones



2. El límite de RAG

Qué preguntas no resuelve y por qué nos importan



4. Explotar el grafo

Pattern matching, expansión y entrega al LLM



6. Un paso más

Function-tools y agentes para RAG avanzado



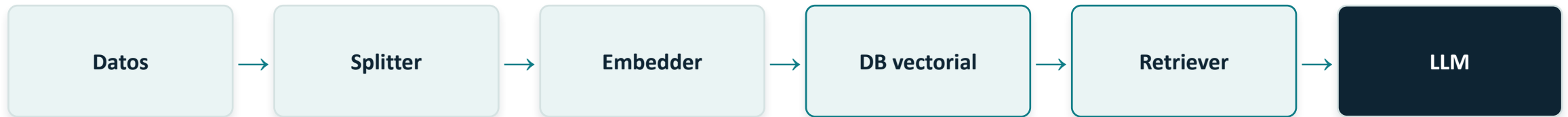
PUNTO DE PARTIDA

Repaso rápido

RAG y grafos



RAG: recuperar para responder mejor



Sirve para búsqueda...

Semántica con vectores densos: cercanía en el espacio de embeddings.

Por palabras clave con vectores sparse / léxico.

Y la combinación híbrida (RRF).



...y la diferencia está en dos lugares

El retriever: cuántos chunks, cómo ranqueo, qué estrategia de búsqueda.

El contexto al LLM: cómo armo el prompt con lo recuperado. El LLM no “sabe”: construye con lo que le doy.

Grafos

De las clases 12 y 13. Cada concepto de grafos mapea a una estrategia de recuperación:



Centralidad

PageRank, grado, intermediación

Ponderar y priorizar nodos por importancia, no solo por similitud.



Navegación / caminos

BFS, DFS, caminos más cortos

Recuperación multi-hop: seguir relaciones entre entidades.



Comunidades

Clustering, detección de grupos

Recuperar y resumir por vecindarios temáticos coherentes.

¿Qué resuelve RAG por sí solo... y qué no?

El top-k vectorial devuelve fragmentos parecidos a la pregunta. Pero hay preguntas donde “parecido” no alcanza:



Multi-hop / relacionales

“¿Quién dirige el laboratorio donde trabaja el autor del paper X?”

La similitud recupera el paper, pero la respuesta exige seguir aristas. Un chunk plano no codifica la relación autor → laboratorio → director.



Globales / agregativas

“¿Cuáles son los temas que conectan todos estos documentos?”

Ningún fragmento contiene la respuesta. El top-k trae k trozos parecidos y se pierde la visión de conjunto que está repartida en todo el corpus.

Lo que ambas comparten: el conocimiento no es solo similitud, es conexión. Ahí entra el grafo.



EL TEMA DE HOY

GraphRAG

Usar la estructura del grafo para recuperar mejor contexto



“GraphRAG” significa dos cosas distintas

A · Construir el grafo desde texto

Partimos de documentos no estructurados. Un LLM **extrae entidades y relaciones**, se arma un grafo de conocimiento, se detectan comunidades y se generan resúmenes.

Es el “GraphRAG” de Microsoft, pensado para preguntas globales.

B · Explotar un grafo existente

Ya tenemos un grafo —datos estructurados o uno que construimos— y **recuperamos sobre él** con pattern matching y expansión de vecindario.

Es lo que haremos hoy y en la tarea con MillenniumDB.

La idea clave: sembrar con vectores, expandir con el grafo

Lo mejor de ambos mundos en un solo pipeline:

1

Búsqueda semántica (HNSW)

encuentra los nodos de entrada más parecidos a la consulta.

2

Expansión por el grafo

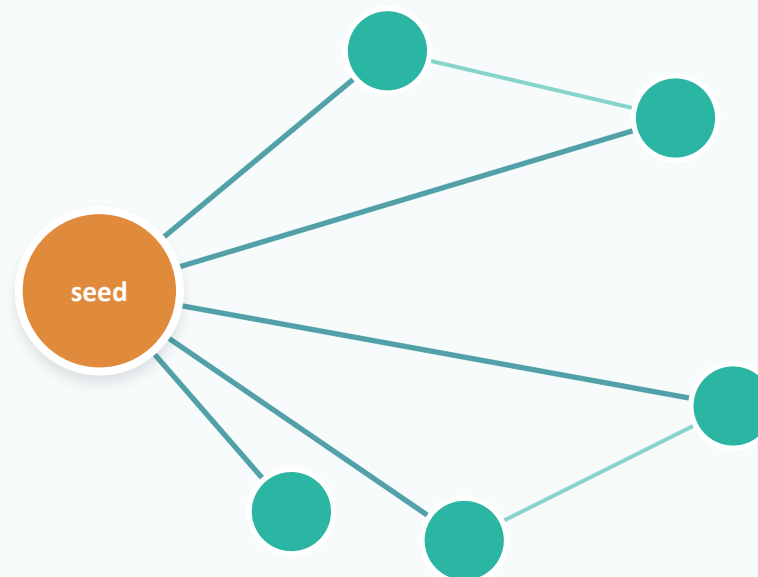
desde esos nodos, navegamos relaciones para traer contexto conectado.

3

Contexto al LLM

entregamos el subgrafo recuperado, con sus relaciones.

consulta



vecinos expandidos

Creación de grafos de conocimiento

Cuando no partimos de datos ya estructurados, lo construimos desde el texto (enfoque A):



Extracción

El LLM lee cada chunk y emite tripletas (sujeto, relación, objeto) y atributos.

Consolidación

Se fusionan menciones de la misma entidad y se unifican relaciones duplicadas.

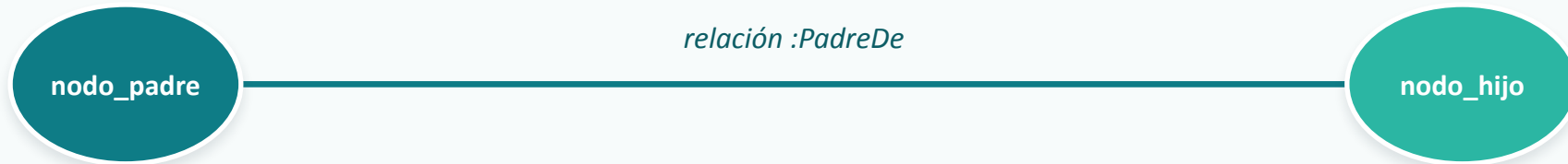
Comunidades

Se agrupan nodos densamente conectados (p. ej. Leiden) y se resume cada comunidad → habilita preguntas globales.

Es más caro de construir y mantener: ese es el tradeoff que veremos.

Explotar el grafo (1): aprovechar la estructura

Recuperar por forma del grafo, no solo por similitud. Describimos un patrón y la BD lo encuentra:



```
// MQL - encontrar hijos de una entidad
MATCH (p :Persona)-[:PadreDe]->(h :Persona)
WHERE p.nombre == "Ana"
RETURN h.nombre
```

Explotar el grafo (2): expansión de vecindario

1

Ciega

Traer todos los vecinos de A. Simple, pero puede inundar de ruido.

2

Filtrada por relación

Solo vecinos por ciertos tipos de arista (p. ej. :CitaA).

3

Ponderada por relevancia

Ranear vecinos por similitud y/o centralidad del grafo.

4

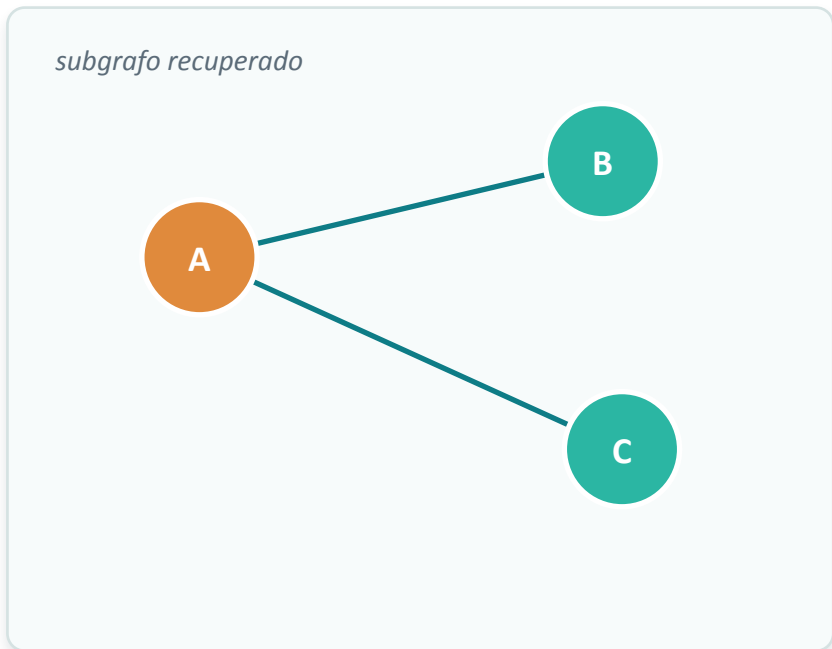
Por comunidades

Traer el vecindario/resumen de la comunidad de A.

Dos perillas que no podés ignorar: **profundidad (hops)** — 1-hop vs k-hop, y la explosión combinatoria; **dilución de contexto** — expandir de más ahoga al LLM en ruido (vuelve el “¿cuántos chunks?”).

Cómo le entregamos el grafo al LLM

El LLM lee texto, no grafos. Hay que serializar el subgrafo recuperado — y aquí está el valor agregado:



```
Contexto (tripletas verbalizadas):  
A -[autor_de]→ B  
A -[trabaja_en]→ C  
C -[dirigido_por]→ ...
```

→ **Verbalizar tripletas:** “A es autor de B”; o serializar el subgrafo como lista de aristas / tabla.

Conservar relaciones: las etiquetas (:dirige, :cita) son la señal extra sobre el chunk plano.

Arrastrar procedencia: nodo/arista → documento fuente, para citar (engancha con el Ejercicio 2).

Otras estrategias: **JSON** - más legible, pero más costoso en términos de tokens

GraphRAG no es gratis: los tradeoffs



Lo que ganamos

- Razonamiento multi-hop siguiendo relaciones.
- Preguntas globales vía comunidades y resúmenes.
- Recuperación con estructura y filtros por relación.
- Procedencia explícita para citar fuentes.



Lo que cuesta

- Construir el grafo: extracción con LLM, costosa y ruidosa.
- Más piezas móviles: grafo + vectores + lógica de traversal.
- Actualizar es más difícil que reindexar chunks.
- Más perillas que afinar (hops, pesos, comunidades).

¿Cuándo basta RAG? ¿Cuándo gana GraphRAG?

Criterio	Basta RAG vanilla	Gana GraphRAG
Tipo de pregunta	Factual / semántica directa	Multi-hop, relacional o global
Estructura de los datos	Texto suelto, poca conexión	Entidades y relaciones ricas / ya estructuradas
Necesito explicar el porqué	Basta con el pasaje	Quiero el camino / las conexiones
Costo y mantención	Bajo, simple de actualizar	Tolero más costo a cambio de alcance

Regla práctica: empezá con RAG; subí a GraphRAG cuando las preguntas pidan conexiones, no solo coincidencias.



LA BASE DE DATOS DE LA TAREA

MillenniumDB

Motor de grafos y RDF del IMFD · sintaxis y operaciones



Qué es y qué nos ofrece

Motor orientado a grafos del Instituto Milenio Fundamentos de los Datos. En desarrollo activo.



Múltiples modelos

RDF + SPARQL 1.1, Quad Model con lenguaje tipo Cypher (MQL), y GQL. (property graphs VS RDF)



Tensores nativos

Embeddings y operaciones vectoriales (p. ej. distancia coseno).



Índice HNSW

Vecinos aproximados sobre los tensores para búsqueda escalable.



Pattern matching

Consultas por patrones de grafo: la base para componer GraphRAG.

Combinando búsqueda vectorial + patrones, MillenniumDB nos da las piezas para armar el retrieval de GraphRAG.

Sintaxis: tensores y similitud exacta

Los embeddings son una propiedad más del nodo. La similitud exacta (sin índice) se calcula con distancia coseno:

Declarar un tensor

```
// MQL
tensorFloat("[1.0, 1.5, 3.2"])
```

Top-K por distancia coseno (búsqueda exacta, sin índice)

```
// MQL
LET ?fixed = emb_198046
MATCH (?a :Embedding)
LET ?d = COSINE_DISTANCE (?fixed.value, ?a.value)
ORDER BY ?d RETURN ?a, ?d LIMIT 100
```

Funciona al instante con pocos tensores; con millones, necesitamos un índice.

Sintaxis: índice HNSW para escalar

```
// MQL - crear el índice (vía update)
CREATE HNSW INDEX "{INDEX_NAME}" WITH {
  "property" = "{property_name}",
  "dimension" = {dim},
  "maxCandidates" = {int},
  "maxEdges" = {int},
  "metric" = "{distance_function}" }
```

```
CREATE HNSW INDEX "idx_congreso" WITH {
  "property" = "value",
  "dimension" = 768,
  "maxCandidates" = 8,
  "maxEdges" = 16,
  "metric" = "cosineDistance" }
```

¿Les suena? Son los parámetros de HNSW de la Clase 11: **maxEdges = M** (conexiones por nodo), **maxCandidates $\approx$ ef_construction** (amplitud al indexar). Más calidad $\leftrightarrow$ más RAM e inserciones lentas.

Componer el retrieval de GraphRAG

MillenniumDB no tiene un botón “GraphRAG”: nos da las primitivas y nosotros las combinamos.



```
// MQL — semilla semántica + expansión por el grafo
MATCH (?vec)
WHERE ?vec==emb_198046
CALL HNSW_TOP_K("congreso", ?vec.value, 8, 100) YIELD ?object AS ?emb, ?distance
MATCH (?parl :Parlamentario)-[:enParticipacion]->(?partic)<-[:tieneParticipation]-(?emb)
ORDER BY ?distance
RETURN ?distance, ?parl.nombre_completo, ?partic.camara, ?partic.texto_principal
```

En la tarea: nos quedamos exactamente aquí — en recuperación, sin pasar todavía a generación.

Actividad: levantar y recuperar en local

Objetivo: cada quien levanta MillenniumDB, la consume y recupera información. Nos detenemos en retrieval (sin LLM).

1

Levantar

MillenniumDB en local
(Docker)

2

Cargar

el dataset con embeddings

3

Consulta semántica

con HNSW_TOP_K

4

Consulta de patrón

MATCH sobre el grafo

5

Combinar

semilla + expansión =
retrieval GraphRAG



Pausa

Volvemos para el cierre del curso





CIERRE DEL CURSO

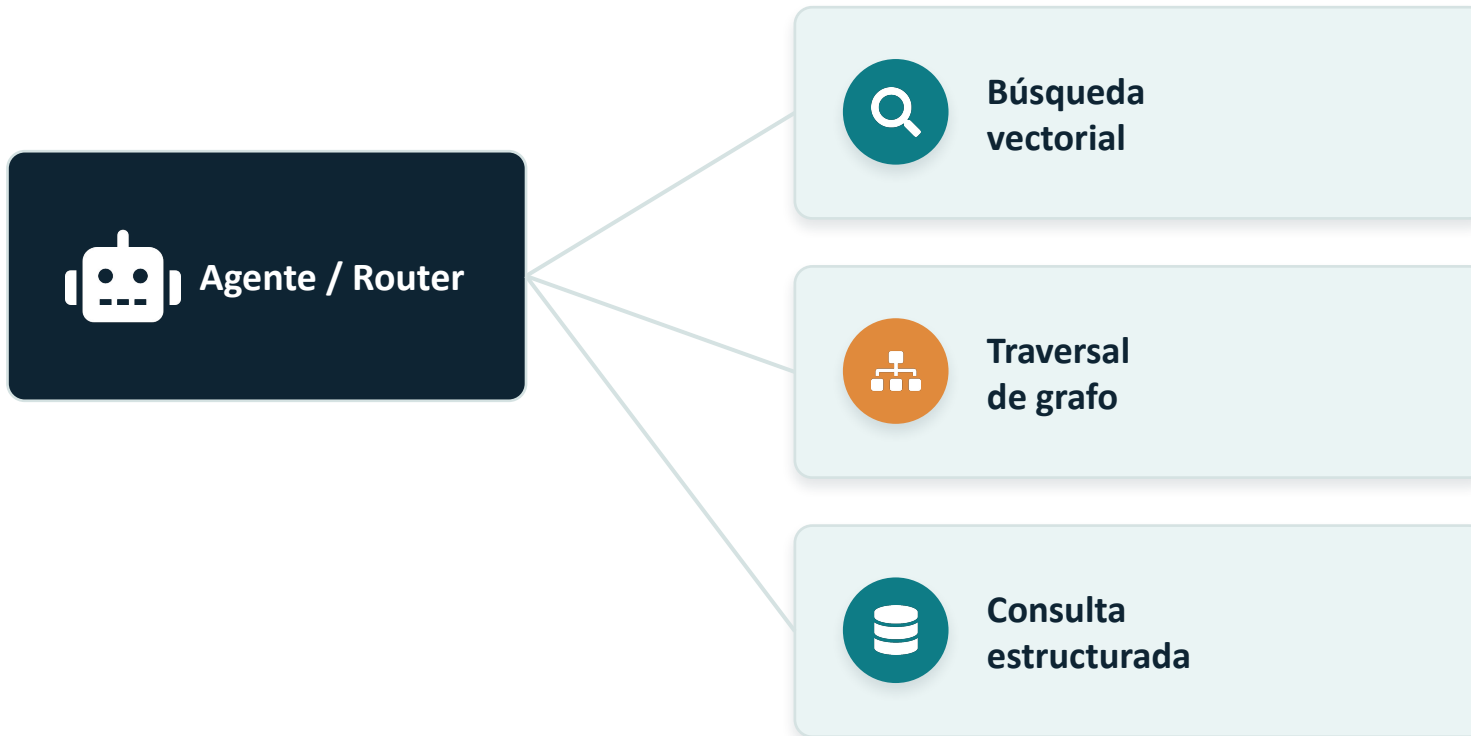
Un paso más allá

Function-tools y agentes: hacia dónde sigue esto



Function-tools y agentes para RAG avanzado

¿Y si el LLM decide cómo recuperar? Exponemos cada recuperación como una herramienta que puede invocar:



El LLM decide si una pregunta pide semántica, relaciones o ambas. De ahí a agentes especializados y multiagente.

Para llevarse

- 1 RAG recupera lo parecido; GraphRAG recupera lo conectado.
- 2 El grafo viene dado (estructurado) o se construye desde texto.
- 3 Se explota con patrones + expansión de vecindario, y se verbaliza para el LLM.
- 4 El patrón estrella: sembrar con vectores (HNSW) y expandir con el grafo.

La duda para seguir avanzando: *¿Cómo evaluamos si nuestro retrieval con grafo realmente mejora las respuestas? ¿Y hasta dónde dejamos que el agente decida solo?*

Lectura complementaria



MillenniumDB — Wiki y “Working with tensors”

`github.com/MillenniumDB/MillenniumDB/wiki`



Microsoft GraphRAG (enfoque A: grafo desde texto)

`microsoft.github.io/graphrag`



HNSW, explicado

`pinecone.io/learn/series/faiss/hnsw`